

AulaNomina – Split 10

Arquitectura funcional + sistema de identificadores jerárquicos

Objetivo del Split

Resolver un problema estructural importante del ERP antes de seguir escalando módulos.

Hasta ahora, el sistema utilizaba IDs internos de base de datos (1, 2, 3, 6, 7...), válidos técnicamente pero poco intuitivos desde el punto de vista funcional y docente.

Se ha rediseñado el sistema para trabajar con **códigos visibles jerárquicos**, más cercanos a cómo funcionaría un ERP real y mucho más comprensibles para alumnos.

Ejemplo anterior:

Trabajador 6
Contrato 11
Nómina 19

Nuevo enfoque:

Empresa 1
Trabajador 1.1
Contrato 1.1.2
Nómina N-1.1.2-2026-05

Mucho más lógico, trazable y fácil de entender.

Problema detectado

El modelo original generaba varios problemas:

1. IDs difíciles de interpretar

No había relación visual entre:

- empresa
- trabajador
- contrato
- nómina
- incidencia

Ejemplo:

Empresa: 2
Trabajador: 7
Contrato: 18
Nómina: 25

No permitía entender rápidamente:

- a qué empresa pertenecía
- qué contrato estaba relacionado
- si un trabajador tenía varios contratos
- el orden cronológico o lógico

2. Poco realista para simulación docente

AulaNomina no es un ERP real de producción.

Es un **simulador académico**, por lo que debe ser:

- visual
- comprensible
- didáctico
- parecido a un ERP profesional

Un alumno debe poder leer un código y entenderlo.

3. Problemas futuros de escalabilidad

Más adelante habrá:

- múltiples empresas
- centros de trabajo
- trabajadores duplicados entre centros
- históricos de contratos
- nóminas
- incidencias

Si no se corregía ahora, la estructura se volvería confusa.

Decisión arquitectónica

Se mantuvieron los **IDs internos reales de base de datos**.

Ejemplo:

```
employee.id = 7  
contract.id = 18  
payroll.id = 25
```

Estos siguen existiendo internamente.

Pero se creó un sistema de:

Códigos visibles funcionales

Separados de la base de datos.

Esto evita romper relaciones SQL y mantiene el backend limpio.

Nueva jerarquía funcional

Empresa

Formato:

1
2
3

Autoincremental.

Ejemplo:

1 → Fundación Diocesana
2 → Colegio Trinidad
3 → AulaNomina Demo

Trabajador

Formato:

`empresa.trabajador`

Ejemplo:

1.1
1.2
1.3
2.1
2.2

Interpretación:

1.2

=

Empresa 1
Segundo trabajador de esa empresa

Contrato

Formato:

`empresa.trabajador.contrato`

Ejemplo:

1.1.1
1.1.2
1.2.1
2.1.1

Interpretación:

1.1.2

=

Empresa 1
Trabajador 1
Segundo contrato

Esto permite representar perfectamente:

- temporales
- conversiones a indefinido
- cambios de jornada
- subrogaciones
- históricos

Incidencias

La incidencia se vincula al contrato.

Ejemplo:

IT → contrato 1.1.2
Vacaciones → contrato 1.1.2

Permite entender rápidamente el origen.

Nóminas

Nuevo formato:

N-empresa.trabajador.contrato-año-mes

Ejemplo:

N-1.1.2-2026-05

Interpretación:

Nómina
Empresa 1
Trabajador 1
Contrato 2
Mayo 2026

Mucho más legible para docencia.

Desarrollo realizado

1. Creación del sistema de códigos visibles

Nuevo archivo:

frontend/src/utils/visibleCodes.js

Se centralizó toda la lógica de cálculo.

Objetivo:

Evitar duplicar código por todo el frontend.

Ahora cualquier módulo puede reutilizar:

Trabajadores

`getEmployeeVisibleCode()`

Contratos

`getContractVisibleCode()`

Incidencias

`getIncidentContractVisibleCode()`

Nóminas

`getPayrollVisibleCode()`

2. Refactorización de tablas

Se actualizaron los listados para mostrar:

Antes

1
7
6
2
3

Ahora

1.1
1.2
1.3
2.1
2.2

Mucho más limpio visualmente.

3. Refactorización de fichas de trabajador

La ficha del trabajador ahora muestra:

Código visible

Ejemplo:

1.2

en lugar de:

7

Contratos

Antes:

ID 18

Ahora:

1.2.1

Nóminas

Antes:

25

Ahora:

N-1.2.1-2026-05

Incidencias

Se muestran asociadas al contrato visible.

Mucho más entendible.

Principio arquitectónico aplicado

Se separan dos conceptos:

ID técnico (backend)

Invisible al usuario.

Sirve para:

- SQL
- relaciones
- foreign keys
- rendimiento
- integridad

Ejemplo:

`employee.id = 7`

Código funcional (frontend)

Visible al usuario.

Sirve para:

- navegación
- trazabilidad
- experiencia docente
- simulación ERP

Ejemplo:

1.2

Este enfoque es el correcto y evita errores futuros.

Resultado

AulaNomina empieza a tener ya comportamiento de **ERP HCM educativo realista**, no solo un CRUD conectado a una base de datos.

La base funcional del MVP ya es sólida para empezar a construir encima sin arrastrar deuda estructural importante.